Shoeb Sheikh (CS23074)

ML Lab-7

Aim: To apply Support vector machine for classification an analayze the impact of different kernels on model accuracy

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import seaborn as sns
%matplotlib inline
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
path = '/content/drive/MyDrive/ML_Classes/pulsar_stars.csv'
df = pd.read_csv(path)
df.head()
```

	Mean of the integrated profile	Standard deviation of the integrated profile	Excess kurtosis of the integrated profile	Skewness of the integrated profile	Mean of the DM-SNR curve	Standard deviation of the DM-SNR curve	Excess kurtosis of the DM-SNR curve	Skewness of the DM-SNR curve	target_class
0	140.562500	55.683782	-0.234571	-0.699648	3.199833	19.110426	7.975532	74.242225	0
1	102.507812	58.882430	0.465318	-0.515088	1.677258	14.860146	10.576487	127.393580	0
2	103.015625	39.341649	0.323328	1.051164	3.121237	21.744669	7.735822	63.171909	0
3	136.750000	57.178449	-0.068415	-0.636238	3.642977	20.959280	6.896499	53.593661	0

```
df.shape
```

```
(17898, 9)
```

```
df.describe()
```

	Mean of the integrated profile	Standard deviation of the integrated profile	Excess kurtosis of the integrated profile	Skewness of the integrated profile	Mean of the DM-SNR curve	Standard deviation of the DM-SNR curve	Excess kurtosis of the DM-SNR curve	Skewness of the DM-SNR curve	target_c
count	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.00
mean	111.079968	46.549532	0.477857	1.770279	12.614400	26.326515	8.303556	104.857709	0.09
std	25.652935	6.843189	1.064040	6.167913	29.472897	19.470572	4.506092	106.514540	0.28
min	5.812500	24.772042	-1.876011	-1.791886	0.213211	7.370432	-3.139270	-1.976976	0.00
25%	100.929688	42.376018	0.027098	-0.188572	1.923077	14.437332	5.781506	34.960504	0.00
50%	115.078125	46.947479	0.223240	0.198710	2.801839	18.461316	8.433515	83.064556	0.00
75%	127.085938	51.023202	0.473325	0.927783	5.464256	28.428104	10.702959	139.309330	0.00

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17898 entries, 0 to 17897
Data columns (total 9 columns):
 #   Column                                        Non-Null Count  Dtype
---  ------                                        --------------  -----
 0    Mean of the integrated profile               17898 non-null  float64
 1    Standard deviation of the integrated profile 17898 non-null  float64
 2    Excess kurtosis of the integrated profile    17898 non-null  float64
 3    Skewness of the integrated profile           17898 non-null  float64
 4    Mean of the DM-SNR curve                     17898 non-null  float64
 5    Standard deviation of the DM-SNR curve       17898 non-null  float64
 6    Excess kurtosis of the DM-SNR curve          17898 non-null  float64
 7    Skewness of the DM-SNR curve                 17898 non-null  float64
 8   target_class                                  17898 non-null  int64
dtypes: float64(8), int64(1)
memory usage: 1.2 MB
```

```
df.columns
```

```
Index([' Mean of the integrated profile',
       ' Standard deviation of the integrated profile',
       ' Excess kurtosis of the integrated profile',
       ' Skewness of the integrated profile', ' Mean of the DM-SNR curve',
       ' Standard deviation of the DM-SNR curve',
       ' Excess kurtosis of the DM-SNR curve', ' Skewness of the DM-SNR curve',
       'target_class'],
      dtype='object')
```

```
df.columns = df.columns.str.strip()
print(df.columns)
```

```
Index(['Mean of the integrated profile',
       'Standard deviation of the integrated profile',
       'Excess kurtosis of the integrated profile',
       'Skewness of the integrated profile', 'Mean of the DM-SNR curve',
       'Standard deviation of the DM-SNR curve',
       'Excess kurtosis of the DM-SNR curve', 'Skewness of the DM-SNR curve',
       'target_class'],
      dtype='object')
```

```
df = df.rename(columns={
    'Mean of the integrated profile': 'MIP',
    'Standard deviation of the integrated profile': 'SDIP',
    'Excess kurtosis of the integrated profile': 'EKIP',
    'Skewness of the integrated profile': 'SKIP',
    'Mean of the DM-SNR curve': 'MDSNR',
    'Standard deviation of the DM-SNR curve': 'SDSNR',
    'Excess kurtosis of the DM-SNR curve': 'EKSNR',
    'Skewness of the DM-SNR curve': 'SKSNR',
    'target_class': 'Target'
})
```

```
df.describe()
```

	MIP	SDIP	EKIP	SKIP	MDSNR	SDSNR	EKSNR	SKSNR	Tar
count	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000000	17898.000
mean	111.079968	46.549532	0.477857	1.770279	12.614400	26.326515	8.303556	104.857709	0.091
std	25.652935	6.843189	1.064040	6.167913	29.472897	19.470572	4.506092	106.514540	0.288
min	5.812500	24.772042	-1.876011	-1.791886	0.213211	7.370432	-3.139270	-1.976976	0.000
25%	100.929688	42.376018	0.027098	-0.188572	1.923077	14.437332	5.781506	34.960504	0.000
50%	115.078125	46.947479	0.223240	0.198710	2.801839	18.461316	8.433515	83.064556	0.000
75%	127.085938	51.023202	0.473325	0.927783	5.464256	28.428104	10.702959	139.309330	0.000
max	192.617188	98.778911	8.069522	68.101622	223.392141	110.642211	34.539844	1191.000837	1.000

```
df.columns
```

```
Index(['MIP', 'SDIP', 'EKIP', 'SKIP', 'MDSNR', 'SDSNR', 'EKSNR', 'SKSNR',
       'Target'],
      dtype='object')
```

```
X = df.iloc[:, :-1]
y = df.iloc[:, -1]
```

```
X.head()
```

	MIP	SDIP	EKIP	SKIP	MDSNR	SDSNR	EKSNR	SKSNR
0	140.562500	55.683782	-0.234571	-0.699648	3.199833	19.110426	7.975532	74.242225
1	102.507812	58.882430	0.465318	-0.515088	1.677258	14.860146	10.576487	127.393580
2	103.015625	39.341649	0.323328	1.051164	3.121237	21.744669	7.735822	63.171909
3	136.750000	57.178449	-0.068415	-0.636238	3.642977	20.959280	6.896499	53.593661
4	88.726562	40.672225	0.600866	1.123492	1.178930	11.468720	14.269573	252.567306

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42
)
```

```
df.columns
```

```
Index(['MIP', 'SDIP', 'EKIP', 'SKIP', 'MDSNR', 'SDSNR', 'EKSNR', 'SKSNR',
       'Target'],
      dtype='object')
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
X_train
```

```
array([[ 0.39398903,  0.27225212, -0.09759786, ..., -0.40338108,
        -0.19481754, -0.37780071],
       [-1.32725743, -1.52127273,  0.21878252, ..., -0.44578706,
         0.23837256, -0.02857034],
       [ 0.76130595,  0.98636783, -0.3245843 , ..., -0.2004973 ,
         0.038556  , -0.29531727],
       ...,
       [ 0.19435365, -0.48635278,  0.17242235, ..., -0.75001383,
         1.92294752,  1.91898259],
       [ 0.95911539,  0.50175114, -0.53401036, ..., -0.24457614,
        -0.20616758, -0.4040674 ],
       [ 0.37725131,  0.58093396, -0.12507761, ..., -0.74159513,
         2.05677923,  2.08022728]])
```

```
y_train
```

	Target
12223	0
1919	0
2753	0
17220	0
12110	0
...	...
11284	0
11964	0
5390	0
860	0
15795	0

12528 rows × 1 columns

dtype: int64

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
svc=SVC()
svc.fit(X_train,y_train)
y_pred=svc.predict(X_test)
accuracy_score(y_test,y_pred)
print('Model accuracy score with default hyperparameter: {0:0.4f}' .format(accuracy_score(y_test, y_pred)))
```

```
Model accuracy score with default hyperparameter: 0.9790
```

```
from sklearn.metrics import confusion_matrix, accuracy_score
```

```
 print('confusion_matrix')
cm=confusion_matrix(y_test, y_pred)
print(confusion_matrix(y_pred, y_test))
```

```
confusion_matrix
[[4863   92]
 [  21  394]]
```

```
print('accuracy_score')
print(accuracy_score(y_pred, y_test))
```

```
accuracy_score
0.9789571694599628
```

```
print('classification_report')
print(classification_report(y_pred, y_test))
```

```
classification_report
              precision    recall  f1-score   support

           0       1.00      0.98      0.99      4955
           1       0.81      0.95      0.87       415

    accuracy                           0.98      5370
   macro avg       0.90      0.97      0.93      5370
weighted avg       0.98      0.98      0.98      5370
```

```
svs=SVC(C=100.0)
svs.fit(X_train,y_train)
y_pred_svs = svs.predict(X_test)
print('Model accuracy score with rbf kernel and C=100.0 :{0:0.4f}'.format(accuracy_score(y_test, y_pred_svs)))
```

```
Model accuracy score with rbf kernel and C=100.0 :0.9803
```

```
linear_svc=SVC(kernel='linear',C=1.0)
linear_svc.fit(X_train,y_train)
y_pred_test=linear_svc.predict(X_test)
linear_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with linear kernel and c=1.0 : {0:.4f}'.format(linear_accuracy))
```

```
Model accuracy score with linear kernel and c=1.0 : 0.9788
```

```
linear_svc=SVC(kernel='linear',C=100.0)
linear_svc.fit(X_train,y_train)
y_pred_test=linear_svc.predict(X_test)
linear_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with linear kernel and c=100.0 : {0:.4f}'.format(linear_accuracy))
```

```
Model accuracy score with linear kernel and c=100.0 : 0.9786
```

```
linear_svc=SVC(kernel='linear',C=1000.0)
linear_svc.fit(X_train,y_train)
y_pred_test=linear_svc.predict(X_test)
linear_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with linear kernel and c=1000.0 : {0:.4f}'.format(linear_accuracy))
```

```
Model accuracy score with linear kernel and c=1000.0 : 0.9786
```

SVM with polynomial

```
poly_svc=SVC(kernel='poly',C=1.0)
poly_svc.fit(X_train,y_train)
y_pred_test=poly_svc.predict(X_test)
poly_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with polynomial kernel and c=1.0 : {0:.4f}'.format(poly_accuracy))
```

```
Model accuracy score with polynomial kernel and c=1.0 : 0.9769
```

```
poly_svc=SVC(kernel='poly',C=100.0)
poly_svc.fit(X_train,y_train)
y_pred_test=poly_svc.predict(X_test)
poly_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with polynomial kernel and c=100.0 : {0:.4f}'.format(poly_accuracy))
```

```
Model accuracy score with polynomial kernel and c=100.0 : 0.9793
```

svm with sigmoid

```
sigmoid_svc=SVC(kernel='sigmoid',C=1.0)
sigmoid_svc.fit(X_train,y_train)
y_pred_test=sigmoid_svc.predict(X_test)
sigmoid_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with sigmoid kernel and c=1.0 : {0:.4f}'.format(sigmoid_accuracy))
```

```
Model accuracy score with sigmoid kernel and c=1.0 : 0.8773
```

```
sigmoid_svc=SVC(kernel='sigmoid',C=100.0)
sigmoid_svc.fit(X_train,y_train)
y_pred_test=sigmoid_svc.predict(X_test)
sigmoid_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with sigmoid kernel and c=100.0 : {0:.4f}'.format(sigmoid_accuracy))
```

```
Model accuracy score with sigmoid kernel and c=100.0 : 0.8773
```

```
sigmoid_svc=SVC(kernel='sigmoid',C=1000.0)
sigmoid_svc.fit(X_train,y_train)
y_pred_test=sigmoid_svc.predict(X_test)
sigmoid_accuracy = accuracy_score(y_test,y_pred_test)
print('Model accuracy score with sigmoid kernel and c=1000.0 : {0:.4f}'.format(sigmoid_accuracy))
```

```
Model accuracy score with sigmoid kernel and c=1000.0 : 0.8773
```

```
print('Confusion Matrix')
cm = confusion_matrix(y_pred,y_test)
print(confusion_matrix(y_pred,y_test))
```

```
Confusion Matrix
[[4863   92]
 [  21  394]]
```